



NALMO: Transforming queries in natural language for moving objects databases

Xieyang Wang¹ · Mengyi Liu¹ · Jianqiu Xu¹ · Hua Lu²

Received: 20 January 2022 / Revised: 20 October 2022 / Accepted: 2 February 2023 /
Published online: 6 May 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Moving objects databases (MODs) have been extensively studied due to their wide variety of applications including traffic management, tourist service and mobile commerce. However, queries in natural languages are still not supported in MODs. Since most users are not familiar with structured query languages, it is essentially important to bridge the gap between natural languages and the underlying MODs system commands. Motivated by this, we design a natural language interface for moving objects, named NALMO. In general, we use semantic parsing in combination with a location knowledge base and domain-specific rules to interpret natural language queries. We design a corpus of moving objects queries for model training, which is later used to determine the query type. Extracted entities from parsing are mapped through deterministic rules to perform query composition. NALMO is able to well translate moving objects queries into structured (executable) languages. We support five kinds of queries including time interval queries, range queries, nearest neighbor queries, trajectory similarity queries and join queries. We develop the system in a prototype system SECONDO and evaluate our approach using 280 natural language queries extracted from popular conference and journal papers in the domain of moving objects. Four volunteers give the system satisfaction and related suggestions through three rounds of independent tests. Experimental results show that (i) *NALMO achieves accuracy and precision 96.8% and 81.1%, respectively*, (ii) *the average time cost of translating a query is 1.49s*, and (iii) *the average satisfaction is 95.5%*.

✉ Jianqiu Xu
jianqiu@nuaa.edu.cn

Xieyang Wang
xieyang@nuaa.edu.cn

Mengyi Liu
liumengyi@nuaa.edu.cn

Hua Lu
luhua@ruc.dk

¹ Nanjing University of Aeronautics and Astronautics, Nanjing, China

² Roskilde University, Roskilde, Denmark

Keywords Moving objects database · Natural language interface · Semantic parsing · Structured language · Query processing

1 Introduction

Mobile computing devices such as PDAs and mobile phones have been widely used in real life. As a result, a large amount of mobile data is collected and stored in the system. Such data represents objects continuously changing locations over time. The system managing such data is called moving objects databases, MOD for short [17]. In the domain of moving objects databases, a number of queries and analysis tasks have been extensively studied including range queries [8], nearest neighbor queries [19], trajectory similarity queries and join queries. Several prototype systems are developed such as HERMES [37], SECONDO [18], PLACE* [54] and TMOM [25].

Despite extensive research having been studied in this area, there is lack of a query interface for non-expert users sending their requests in natural languages. It is a non-trivial task for most users to write structured languages such as SQL. Developing a natural language interface for databases (NLDBs) has become an urgent task as databases support an increasing number of applications. NLDBs will significantly benefit users as they can use natural languages to request a variety of queries instead of mastering the structured language as well as the underlying structure such as the database schema. Figure 1 depicts the task that we would like to achieve to empower MODs. That is, a natural language query is precisely and correctly translated into a structured and executable expression in MODs.

NLDB is a challenging task mainly due to two difficulties: (i) *semantic understanding and analysis*, and (ii) *query translation*. The questions asked by users are usually ambiguous or in-complete in terms of semantic. In the domain of moving objects databases a variety of queries have different key entities including time, location and moving objects identification. Constructing an effective semantic parser for identifying and extracting entities is a crucial task. Although MODs are already equipped with a rich set of operators and access methods,

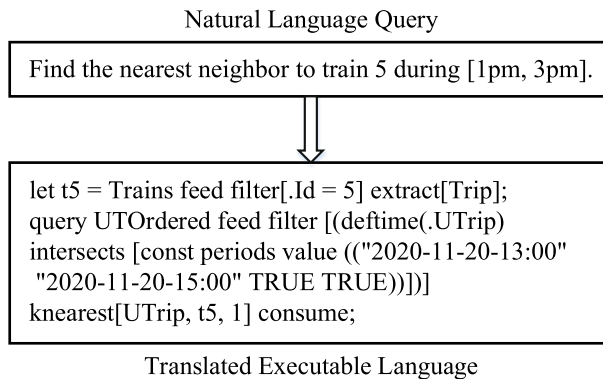


Fig. 1 Example of translating

effectively and accurately combining entities and operators into structured languages is a non-trivial task due to the query diversity.

NLIDB systems have achieved satisfactory results in specific domains. Rule-based and machine learning-based methods are usually used to perform the translation from natural language query to SQL. Some NLIDB systems use machine learning [45] to understand and translate natural language queries such as *NL₂CM* [2] and *SpatialNLI* [28]. *PRECISE* [40], *NaLIR* [27] and *ATHENA* [43] use rule-based semantic analysis and query construction methods. These methods limit expressive power and can only be applied to specific domains. Natural language queries often contain ambiguities and grammatical errors which make mistakes and produce erroneous output. However, a purely rule-based method will greatly restrict the expression of queries that users can put forward. Natural language is inherently ambiguous, and helping NLIDB systems understand query semantics is challenging. All users can do is modify the query and re-query. Some NLIDBs ask users for intervention to perform query reformulation, such as *NaLIX* [31] and *NaLIR*. The users of NLIDB are usually not experts, and when the system gives users answer, they may have difficulty in judging whether it is correct. Some researchers have extended the capabilities of existing systems in terms of explaining both answer and non-answer to queries. *NLProv* [11] is able to provide natural language explanations for query answer. *NLProveNAns* [10] augments *NaLIR* by supporting interpretations of non-answer. State-of-the-art systems based on rule and machine learning methods have been able to translate natural language queries relatively well in different domains.

However, there is no NLIDB that supports MODs. Providing a user-friendly interface for non-experts from other domains is an important research direction of moving objects databases. An interactive interface is preferred, and users can check the correctness and accuracy of translated expressions [35]. Besides, most of the existing benchmarks are used for testing natural language queries of relational databases. For example, the ATIS [42] benchmark for air travel is based on a relational database called Official Airline Guide. The GeoQuery [46] benchmark for the geographic domain is based on the United States geographic database. The Scholar [20] benchmark consists of 816 natural language questions tagged with SQL for academic database search. Corpora that can be used for query classification of MODs are rare or even not available. There is no dataset for moving objects queries, and non-moving object queries cannot be used as test cases to determine the effect of the translation queries to MODs. Most current traditional database management systems including Oracle, MySQL, and ACCESS cannot handle time-varying, multi-dimensional, and variable-length data. The main reason is that spatial information including points, lines, and regions cannot be stored in the traditional two-dimensional table form. If we want to represent a moving object with spatial, temporal and attribute information and answer queries about its location, the location of the moving object needs to be updated continuously. For relational databases, it is impractical to store spatial data and update location information frequently. Compared with relational databases, MODs provide users with many basic operators (e.g., *filter*(●)) and special operators (e.g., *deftime*(●)) for different queries. Without these operators, natural language queries for moving objects are difficult to be effectively translated into corresponding relational database queries.

In this paper, we design and develop a natural language interface for MODs including (i) *semantic parsing and analysis*, (ii) *structured language composition*, and (iii) *selection of suitable datasets*. At the moment, the system supports five kinds of queries including time

interval queries, range queries, nearest neighbor queries, trajectory similarity queries and join queries, but not limited to the five types. This is because the corresponding operators and functions are implemented. Compared with [51], we further support join query. The corpus and test dataset of join query are also constructed. If the operators and functions of other queries and analysis are well handled in the system, those queries and analysis can be well supported. To support natural language queries, the first issue is semantic understanding. We preprocess user-specified spatio-temporal queries in which there may exist punctuation that can affect the processing result or be unnecessary. These punctuations should be replaced with spaces, before subsequent work can be carried out. After preprocessing, word segmentation and entity recognition are performed by appropriate natural language processing tools. The extraction of key semantic information is one of the most important parts of natural language translation. We propose a location knowledge base and a domain methodology for entity analysis such that a word like “*Nanjing*” can be parsed as a location. We extract the city’s POI information through OpenStreetMap (OSM) and allow users to supplement POI information to improve the location knowledge base [51].

We collect more than 60 conference and journal papers to extract query examples, and manually generate moving objects queries for knowledge enhancement to build a moving objects query corpus. The model trained through the corpus can accurately determine the query type. We further refine the scoring criteria of experts for moving object queries [51]. After the type determination, query interpretations are mapped into query fragments based on inference rules, before finally being assembled into complete queries. We implement our proposal in the system SECONDO that provides users with many basic operators and special operators. According to different query types, the parsed entities are mapped to corresponding query fragments and these fragments are combined with operators to obtain the final structured query. Efficiency is also a significant part of the user experience. In order to improve the translation efficiency of the system, we use a location-based prefix index in the process of extracting location entities. It is worth noting that most of the query objects in our examples are public transportation, because one of our test datasets is moving underground trains. However, we can still query moving objects excluding public transportation as long as there are corresponding datasets. We conduct a separate experimental test of the quality of natural language understanding, and ask four volunteers to participate in the user interaction tests. Finally we expound system development, especially how to integrate NALMO into SECONDO [51]. Three criteria are defined to evaluate the translation quality: (i) *accuracy* and *precision*, (ii) *time cost* and (iii) *usability*.

The main contributions of this paper are summarized as follows.

- We design an algorithm for spatio-temporal data parsing in which a location knowledge base is utilized to extract location information. We describe the methods to expand the location knowledge base. A location-based prefix index is developed to improve the extraction efficiency.
- We propose a corpus for query type determination and develop methods for constructing structured query languages which combine the parsed key entities and system operators. We show detailed scoring rules for query extraction, including conference, journal papers and expert scoring rules.
- We develop NALMO in an extensible database system SECONDO and conduct experimental evaluation using two datasets. We introduce how to integrate NALMO in

SECONDO. The experiment results demonstrate that our method achieves accuracy and precision of 96.8% and 81.1%, respectively. The average time cost of translating a query is 1.49s. Through the three rounds of independent testing conducted by four volunteers, the user satisfaction is 95.5%.

The rest of the paper is organized as follows. We review the related work in Section 2 and provide the framework of NALMO in Section 3. We introduce natural language understanding and the query translation in Sections 4 and 5, respectively. The experimental evaluation is presented in Section 6. We introduce how to integrate NALMO in SECONDO in Section 7, followed by the conclusion in Section 8.

2 Related work

MODs have been extensively studied in a range of applications including transportation system, express delivery services and mobile commerce [48, 58]. In the literature, a number of techniques are proposed such as data models [16, 21, 55], index structures [6, 38, 47, 49], query algorithms [8, 19] and algorithms of location selection problems [32]. However, there is still lack of a natural language interface in moving objects databases which supports transforming natural language expressions into executable languages.

2.1 Challenges of NLDBs

Natural Language Interfaces (NLI) have a long history in the field of natural language processing and the database community, and the implementation of NLIDB has also received extensive attention from researchers. NLIDB does not require users to master any underlying knowledge and query language of the database, helping users to realize the function of querying the database through natural language description. Unsurprisingly, the advent of the big data era has made NLIDB more attractive than ever. However, despite years of research, NLIDB remains largely an open research problem, with no general commercial solutions or prototype systems as yet. Two major open challenges hinder the wide adaption of NLDBs: natural language understanding and query translation [29].

2.1.1 Natural language understanding

To have good NLDBs, we need to understand natural language problems well. A number of systems use rule-based methods to understand natural language in a specific domain. These systems are usually restricted in natural language support such as grammar and vocabulary to improve the accuracy of semantic understanding [56]. *PRECISE* [40] defines the notion of semantic tractability and identifies a subset of natural language queries that can be precisely translated into SQL. *NaLIX* [31] limits natural language queries to a controlled subset based on a predefined grammar. To understand the problem of mixing individual and general knowledge, *NL₂CM* [2] uses the IX detector, implemented through declarative selection patterns combined with specialized vocabularies, to distinguish individual query parts from general ones. An ideal NLIDB enables users to make arbitrarily complex temporary queries on the underlying database and obtain accurate information with the least effort. Therefore, in the

process of semantic understanding, *NALIX* and *DiaSQL* [15] modify the query in subsequent user interactions to reformulate the parse tree, but the reformulation usually needs excessive user interactions. *NaLIR* [27] detects the parse tree so that users can directly modify the parse tree instead of the questions raised by the user. It can generate suggestions for users to replan queries when they are outside the semantic scope. *ATHENA* [43] constructs a translation index through specific entity extraction, and uses ontology-based domain knowledge to bridge the semantic gaps caused by leaky abstraction. *ATHENA++* [43, 62] further translates complex nested SQL queries. *NL2TRANQUYL* [5] stores and models relevant information by employing ontology which contains different concepts. *QUERIX* [61] analyzes the syntax of natural language by a syntactic analyzer, which requires that the natural language components must be complete. Otherwise, the results obtained by the syntax analyzer are not accurate enough, which may decrease the accuracy of final results.

With the increasing use of statistical learning techniques, attempts have been made to perform semantic analysis on sentences in the form of various supervision. Pasupat and Liang [36] use question and answer pairs as supervision to solve the answering task of complex questions on semi-structured tables. Poon and Domingos [39] is the first attempt at learning a semantic parsing model in an unsupervised manner [24]. Artzi and Zettlemoyer [3] learn what the user's utterances represent by obtaining feedback from the conversation. Wang et al. [52] successfully realize the training of a semantic parser in a domain without training examples. Their framework consists of two parts: a builder and a domain general grammar. Wong and Mooney [53] use statistical machine translation technology to complete semantic parsing tasks.

In recent years, neural networks [20] are increasingly used to train models of parsing natural language sentences, encoding and decoding for natural language semantic understanding. Although frameworks based on encoder-decoder frameworks offer greater flexibility, they generally do not allow interpretation and understanding of meaning combinations [24]. The approach adopted by Cheng et al. [7] builds the intermediate structure within two stages, allowing for a very clear view of what the model is learning. Similar to this, an intermediate template is also generated in [12], where the final output is sketchy for easier decoding. Yin and Neubig [57] alleviate the need for models to discover inherent syntax from scarce training data by explicitly providing constraints for decoders in the form of target language syntax. The work of the model can then be better focused on building parsing, guided by the known grammar rules.

2.1.2 Query translation

On the basis of a positive understanding of natural language, how to translate natural language is also of crucial importance. The common method of query construction is to use a unique algorithm to construct a structured query from the meaning of natural language query and the domain knowledge corresponding to the underlying database [30]. *ATHENA* proposes a unique two-stage approach that first translates an input natural language query into an ontology query language (OQL) query defined over the ontology, and then translates the OQL query into its corresponding SQL query. *NLPQC* [44] accepts queries based on specific domain templates. *QUERIX* extracts specific components from the syntax tree to match knowledge with the knowledge base to obtain the final result. Up-to-date machine learning technologies are gradually applied to natural language translation. *NL₂CM* [2] mines

the crowd for individual by translating questions of the audience into OASSIS-QL which is defined as an extension of SPARQL. *SpatialNLI* [28] uses seq2seq translation to enhance its translation model. A number of systems combine deterministic algorithm approaches and machine learning methods to construct structured languages [33]. A parser generates corresponding SQL clauses, then further constructs SQL query candidates according to the corresponding templates and rules, and finally ranks the candidates of the constructed languages through the SVM ranker [14].

2.2 User interaction in NLIDBs

Based on the technical approach they use, NLIs can be divided into four categories: keyword-, pattern-, parsing- and grammar-based NLI [1]. User interaction is involved in each type. *QUICK* [59] is a keyword-based NLI, which refines user interaction on the basis of keyword search to enhance the expressiveness of semantic queries. In concrete implementation, after the user raises a keyword question, the system guides the user to select the intention of the question through progressive refinement steps. *NLQ/A* [60] is a pattern-based NLI to query a knowledge graph [23]. The system optimizes the user interaction part to better resolve the ambiguity problem. As a parsing-based NLI, *NaLIR* is an improved version of *NaLIX*. It not only explains to the user how to process the query, but also shows a variety of understandings for the user to choose from to ease the user's burden of eliminating misunderstandings. In grammar-based systems, take *SPARKLIS* [13] as an example. To ensure the completeness of user input when searching for concepts, entities or modifiers, the system uses a cascade of three stages.

These NLIDBs that contain user interaction provide guidance and security to users through intermediate answer and suggestions. When the user inputs a query question, the user is guided to input the correct query statement by demonstrating some available query statements and prompting the types of statements that can be queried. When the question raised by the user is semantically ambiguous, the correct semantic information is determined by providing the user with possible explanations to choose from. When the information entered by the user cannot be retrieved in the database, the system can retrieve whether the user has entered similar information existing in the database, and provide an association prompt for incorrect input, thereby helping the user to complete the query correctly. However, too many interactions and constraints not only reduce the translation speed of the system, but also reduce the user experience. Considering that the main purpose of user interaction is to improve system effectiveness, researchers have gradually begun to use existing data to improve system effectiveness. For example, *TEMPLAR* [4] uses SQL query log information to improve the ability of existing NLIDB to perform keyword mapping and join path inference, greatly improving the accuracy.

2.3 NLIDBs for spatio-temporal databases

NLIDB for spatio-temporal databases has also received attention in recent years [9]. *TEQUILA* [22] proposes a knowledge graph to deal with temporal questions, where cues for temporal relations need to be discovered and handled. Wang et al. propose a spatial understanding model *SpatialNLI* based on contextual semantics [28], which can recognize

the meaning of spatial entities. Spatial terms are extracted from natural language description through symbolic expressions to represent spatial features and relations between them [41]. *NL2TRANQUYL* [5] is a system primarily for trip-planning requests in a large multimodal transportation system with different constraints such as taking the least time, traveling the shortest distance and choosing the least expensive route.

NALMO is designed for queries over moving objects. Some NLDBs [5, 27] use different parsers to obtain dependency parser tree for a given query, and the quality of parser tree depends largely on the parser. *ATHENA* uses the TIMEX annotator to find all time ranges involved in NLQ, the Stanford Numeric Expressions annotator to find all tokens containing numeric quantities, and the Stanford Dependency Parser to identify dependencies between dependants and dependees. *NaLIR* uses Stanford Parser [26] to generate language parsing trees from natural language queries. For MODs, the location and time information is very critical. We do not rely much on natural language processing tools and will further process natural language queries to extract key entities. Majority of the existing NLDBs construct queries via query composition. These NLDBs all map the processed key information to the corresponding structured components. However, for various queries, different DBMSs and structured languages provide diverse operators and clauses, requiring different mapping values. *ATHENA* uses the mapping between the ontology and the database schema to translate the intermediate query language on the ontology into SQL. *NaLIR* needs to determine whether the query tree contains function nodes or quantifier nodes, and then construct it according to the situation. NALMO considers multiple query types in moving objects and designs different mapping templates for each query type, while others mainly use parse tree nodes and ontology elements for mapping information.

Most natural language interfaces of semantic web rely on the use of general dictionaries for resource matching, and WordNet [34] is also used for dictionary expansion. However, general dictionaries are far from meeting the needs of users in certain domains, and thus, specific domain dictionaries should be added. What is more, the lack of benchmark and datasets of NLDBs also catches much attention. There are already a few datasets such as MAS [27] in *NaLIR* which is suitable for academic papers, authors, conferences and universities. GEO [40] in *PRECISE* is for spatial queries but without time information. None of those works deal with natural language queries in moving objects databases. NALMO proposes a dataset containing multiple moving object queries.

3 The framework

We provide an overview of NALMO in Fig. 2. Key components are (i) a *semantic parser*, (ii) a *location knowledge base*, (iii) a *location-based prefix index*, (iv) a *corpus for query*

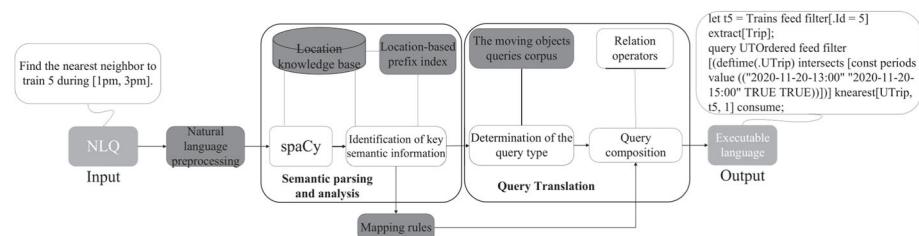


Fig. 2 The NALMO architecture in SECOND0

type determination and (v) *a rule-based algorithm for structured language generation*. At first, the module of natural language processing performs semantic analysis and extracts key entities from the query expression. Next, the query type is determined by a machine learning model and there are five kinds of queries in total, summarized in Table 1. Finally, NALMO maps the entities to queries and combines moving objects operators with entities to produce structured queries. Frequently used notations are summarized in Table 2. Using the query Q5 in Table 1, we illustrate the workflow as follows.

Semantic interpreter We use a semantic parser to identify and extract key entities. We preprocess the punctuation of natural language and leverage the tool *spaCy* for word segmentation and entity recognition. The *spaCy* parser enables NALMO to realize that token numbers (e.g., five and 3) and time (e.g., 8am and 10am) are attached to labels CARDINAL and TIME, respectively. Then, we develop a rule-based algorithm to further extract the location, time (e.g., 8am and 10am), the number of nearest neighbors (e.g., five) and the moving objects identifier (e.g., 3) from parsed entities.

Location knowledge base and location-based prefix index In addition to moving objects, the database also stores relevant point and region objects representing interesting places, e.g., POIs. We find out those objects and put them into the location knowledge base that serves as the dictionary. If the query location does not exist in the dictionary, the location information misses and users will not get the expected results. As location description usually contains a large number of prefixes, we use prefix indices to increase the efficiency of matching between natural language queries and the location knowledge base.

Query type determination We build a corpus for moving object queries in natural languages and utilize an LSTM neural network to train a model for predicting the query type. For example, in Table 1 the expression Q5 “Which trains are the nine continuous nearest neighbors to the train 9 from 8am to 10am.” will be reported as a nearest neighbor query.

Structured language generation NALMO maps the identified key information to the corresponding module in moving objects queries and combines them with defined operators

Table 1 Query examples and types

Id	Natural Language Query Example	Type
Q1	Where did the trains go at a certain time t_1 ?	Time Interval Query
Q2	Do you know where did exactly the train O_1 go during the time period $[t_2, t_3]$?	Time Interval Query
Q3	Please tell me the time that the train 7 arrive at Nanjing Station?	Range Query
Q4	Find the trains that pass the station Nantong during the time interval $[t_4, t_5]$?	Range Query
Q5	Which trains are the nine continuous nearest neighbors to the train 9 from 8am to 10am.	Nearest Neighbor Query
Q6	Show me k nearest neighbors to the train O_2 at any time instance of the time period $[t_6, t_7]$.	Nearest Neighbor Query
Q7	Tell me the trajectory that is similar to the train 7 at any time instance of the time period $[t_8, t_9]$.	Trajectory Similarity Query
Q8	Find the same trajectory between train O_3 and train O_4 during the time period $[t_{10}, t_{11}]$.	Join Query

Table 2 Frequently used notations

Name	Abbreviation
A natural language interface for moving objects	NALMO
Natural language interfaces to database	NLIDB
Moving objects database	MOD
Moving objects queries	MOQ
Berlin trains	BT

to construct structured queries. The expression $Q5$ is a nearest neighbor query, and thus corresponding operators (e.g., $nearestneighbor(\bullet)$) are used to form the expression.

4 Natural language understanding

The task is to precisely capture the meaning of a natural language expression and accurately translate the expression into the query language. Database queries have domain specific features or key attributes, which help analyze the semantics. Effectively extracting those information is essential in understanding natural languages. The procedure is as follows. We retain or delete the punctuations, and utilize the tool *spaCy* to perform word segmentation and do entity recognition.

4.1 Natural language processing

NALMO deletes character interference items such as double quotation marks (“ ”), single quotation marks (‘ ’), exclamation (!) and the right square bracket (]). The left square bracket ([) should be kept because the symbol guarantees the correct identification of time.

After the deletion, we use the industrial-grade *spaCy* parser for word segmentation and entity recognition. Time symbols are labelled TIME, and number symbols are labelled CARDINAL. These time and number symbols are stored for further identification. Numbers may be in the form of English and must be converted to Arabic numbers. If a number is combined with a connector, we replace the connector with a space. For numbers with space such as “one hundred”, we use the following regular expression to cut the space and identify each word.

$$re = re.split('[]', n) \quad (1)$$

After removing the conjunction word “and” in numbers, we traverse the remaining English digits in reverse order to obtain the digits such as ten, hundred and thousand. These numbers are eventually converted to Arabic numbers.

4.2 Identifying key semantic information

Since entities are approximately processed by *spaCy*, key information for moving objects queries is not accurate and the results are not sufficient for structured language construction. To increase the translation quality, we propose an algorithm to further parse the spatio-temporal data in Algorithm 1.

Algorithm 1 Semantic parsing.

Input: query expression, Q ;
prefix tree of location knowledge base, $TrieLDB$;
relation knowledge base in database, RKB ;

Output: the parsing result, Q'

```

1:  $doc \leftarrow nlp(Q)$ 
2:  $tmp\_token \leftarrow find(doc, "neighbor")$ 
3: for  $token_i \in doc$  do
4:   if  $token_i.label\_ = CARDINAL$  then
5:      $N\_list.append(token_i)$ 
6:   end if
7: end for
8: for  $token_i \in doc$  do
9:   if  $token_i.label\_ = TIME$  then
10:     $Q'.time \leftarrow token_i$ 
11:   end if
12:   if  $token_i.pos\_ = NOUN \vee token_i.pos\_ = PROP$  then
13:     if  $search(TrieLDB, token_i.text)$  then
14:        $Q'.location \leftarrow token_i$ 
15:     end if
16:     if  $token_i.text \in RKB$  then
17:        $Q'.relation \leftarrow token_i$ 
18:       if  $token_{i+1} \in N\_list$  then
19:          $Q'.object\_id = token_{i+1}$ 
20:       end if
21:     end if
22:   end if
23:   if  $token_i = tmp\_token$  then
24:      $Q'.number\_NN \leftarrow token_{i-1}$ 
25:   end if
26: end for

```

Given a time word with colon such as “8:00 o'clock”, we use the following expression for matching and extraction.

$$r' \backslash d + [::] \backslash d + ' \quad (2)$$

The contents in “[:]” are optional characters. We take both Chinese and English colons into account as users may type them in a mixed way. A Chinese colon may appear in an English sentence, causing the time extraction error. If the time in the query includes a Chinese colon, the colon is replaced by an English one. For the hour without a colon such as “8am”, we use the following expression for matching and extraction.

$$r' \backslash d + ' \quad (3)$$

Using a regular expression for matching a full hour is simple, but the result cannot be used in the database. The full hour needs to be attached with “:00”. That is, “8am” will be rewritten as “8:00” and the terms “am” and “pm” will be converted to 24-hour format. We traverse the word segmentation results and evaluate each segmented word using the location knowledge base to determine whether the query location exists or not. Query relations are extracted from the database. We analyze the objects in the database and find the relation containing an attribute *mpoint*. The data type *mpoint* is for representing moving objects. This relation has target moving objects. There may be an atomic flow containing only one object with the attribute *mpoint* or a relation that may have multiple moving objects, and the latter is required. An example is provided in Fig. 3 in which there is a relation with the schema *Trains*(*Id:int*, *Trip:mpoint*). *Trains*() is suitable for time interval query and range

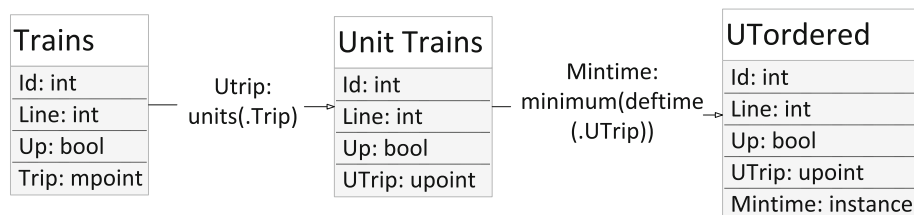


Fig. 3 Example of extracted relations in Berlin Trains dataset

query. Based on the extracted relation, we can extend the existing relation for different query types. *UnitTrains()* adds a new unit attribute *UTrip* on the basis of *Trains()*. *UTordered()* is sorted by start time on the basis of *UnitTrains()*. The two relations are applied to nearest neighbor query and similarity query. We use the relations in Berlin Trains dataset as examples, but not limited to these relations.

The semantics of numbers may be ambiguous in some cases as the number could be the identification of a query object or the number of neighbors. For example, in *Q5*, the moving objects relation in the database is *train*, and the number 9 exists in the number list. Thus, 9 will be an object identifier. In an English query, the number of neighbors must appear before the word “neighbor”, or the user may only query the nearest neighbor without the number of neighbors in the sentence. We utilize this knowledge to determine the semantics.

The analysis of time complexity Assuming that the number of words contained in a natural language query is n , the length of a location string is l , and the number of relations in the database is m , it can be seen that the time complexity of finding location by prefix tree is $O(l)$ (line 13), and the time complexity of determining the query relation is $O(m)$ (line 16). If l is greater than m , the time complexity of Algorithm 1 is $O(n \cdot l)$. Otherwise, the time complexity is $O(n \cdot m)$.

4.3 Location knowledge base and index

Spatial information needs to be captured in moving objects queries. In order to determine the locations, we generate a location knowledge base for each database which contains all the locations included in the database. The location knowledge base extracts objects whose attribute is *point* or *region*, because in *SECONDO* only objects whose attribute is *point* or *region* can be locations. If the location does not exist in the location knowledge base, we generate corresponding structured languages, but users will not get the expected results.

Location information is important for the construction of structured queries of moving objects, and it is necessary to capture spatial information in moving object queries. In order to determine the required location information, the system can automatically generate a location knowledge base for each database, which contains all possible locations in the database. When the dataset is modified or replaced, the system can still generate the corresponding location knowledge base for the database. The test dataset is Berlin train timetable dataset and the following location extractions are based on Berlin and other cities in Germany. According to the three attributes of POI information name, latitude and longitude, and POI description, the POI information collection of Berlin is extracted from OSM. At the same time, we carry out POI information extraction for other cities in Germany. When the required dataset is selected, the location knowledge base will automatically determine the city of the dataset and add the corresponding city POI information to the dataset. If the information already exists, we can

ignore the location. The location information in the location knowledge base mainly contains the following three sources: (i) objects with attributes of point, line and region existing in the dataset; (ii) POI information of the corresponding city extracted by OSM; (iii) user queried location information which does not exist in the constructed location knowledge base and the existing POI collection of other cities. User can give feedback and the administrator can manually add the location information.

In order to improve the query efficiency, we construct a location-based prefix index for matching the location knowledge base. A large number of locations often contain the same prefix. Therefore, the index can effectively improve the extraction efficiency of location information. For example, “Nanjing” (Q3) and “Nantong” (Q4) have the same prefix “Nan”. The root node of the prefix tree is empty, and every child node contains an English character. From the root node to a certain leaf node, the characters passing on the path are connected, which is the location corresponding to the nodes. The characters contained in all child nodes of each node are different from each other. Characters that repeat continuously from the first character occupy only one node. Besides, we merge some nodes with only one child node as the leaf nodes to reduce memory consumption. In the process of prefix tree construction, whenever there is a location, we will start matching from the root node. If there is no matching node for the character, we will create a new node and continue to match the next character. If the node already exists, we follow the path to continue matching. An example of a location-based prefix tree is shown in Fig. 4. With such prefix trees, locations can be determined efficiently.

5 Query translation

The translation consists of two steps: (i) *determining the query type* and (ii) *constructing the structured language*. Semantic understanding and parsing of natural languages produce key entities and map them to structured query languages. There are different types of queries and a corpus is built to determine the query type. The corpus makes use of an LSTM neural network to train a model for identifying the query type. The natural language query needs to be proposed according to a certain grammar, and the moving objects query must contain the key information. Therefore, the corpus is more important for the result of query type determination, and the choices of neural network classifiers have less influence. NALMO selects the mapping method and combines the affluent operators to form structured queries.

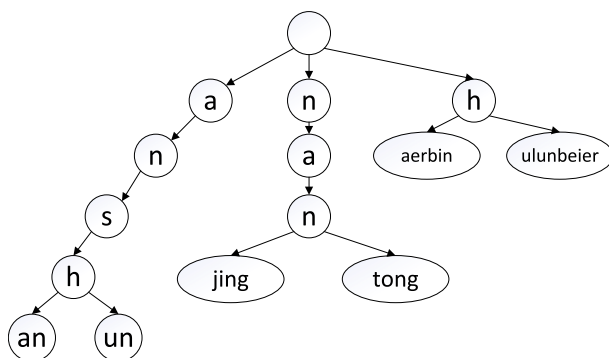


Fig. 4 Example of location-based prefix tree

5.1 Corpus: determining query type

Providing a common entity mapping and query combination method is a challenging task because moving object queries contain specific key information which require different operators. One solution of determining the query type is to use defined rules, but the strategy will produce a set of fixed outputs. When the natural language expression changes, the method probably produces incorrect results. The determination effect primarily depends on the quality of natural language processing tools.

To have high quality processing, we build a moving object corpus. The corpus contains 1500 moving objects queries classified into five categories including (i) time interval query, (ii) range query (iii) nearest neighbor query, (iv) trajectory similarity query, (v) join query. It is worth noting that time interval query includes time point query. Range query includes spatial query and spatio-temporal range query. In the process of classification and training, we also consider the special cases of time interval query and range query. We initially collect 350 moving objects query examples from more than 60 published conference and journal papers, and then use experts' knowledge, entity information replacement and sentence expression replacement to enhance the knowledge. Experts utilize various key information with different semantic expressions according to the query type. These queries satisfy the diversity of semantic expression properly, and the corpus is extended up to 1500 moving objects queries. The experts participating in our work specialize in database, specifically MODs. We manually mark each query.

Data extraction is guided by two criteria: (i) *knowledge of moving objects experts* and (ii) *citation-based impact factors from the Institute for Scientific Information (ISI)*. We use popular academic conferences and journals as references, including 34 journal papers and 28 conference papers. Experts judge the quality of natural language queries in articles and rank these queries. ISI calculates the impact factor of the journal based on its citation index. The assumption is that articles in higher-quality journals and conferences are likely to provide comprehensive queries. Each journal and conference receives a ranking score. The highest ranking journal or conference receives a score 1, and the next highest ranking receives a score 1/2. The ranking score is used as a reference, and the final score is given by the expert. Articles published in the same journal have the same scores. Domain experts rank query examples according to the query robustness, and the maximum score is 1. Then, the article score and the expert score are summarized as the total score, and moving objects queries ranking from high to low will be added in the corpus. Journal and conference scores are given in Table 3.

Experts will also score the different query examples. The scoring criteria of experts are as follows (the query results are accurate to two decimal places). Experts should determine whether the query is semantically complete and whether there are obvious grammatical errors. If the query semantics is incomplete or there are grammatical errors, the score will be 0, and the subsequent judgment will be terminated. Experts judge whether the query belongs to the natural language query of the moving object. If not, the score will be directly 0, and the subsequent judgment is terminated. This module accounts for 0.2 points. Experts judge whether the query is a moving object query type supported by the system. As long as the query belongs to any type of moving objects query, the query gets the score. If the expert cannot determine, they can give scores as appropriate. The query type that is not supported by the system will directly be 0, and the subsequent judgment will be terminated. This module accounts for 0.4 points. Experts judge whether the query contains the entity required by the corresponding query type. If the required entity is missing or the entity expression is ambiguous, the score of the query will be deducted. This module accounts for 0.4 points. An example of expert scoring is shown in Table 4.

Table 3 Scores of journals and conferences

Journal and Conference Proceeding	Score
ACM Conference on Management of Data	1
IEEE Transactions on Knowledge and Data Engineering	1
Proceedings of the VLDB Endowment	1
IEEE International Conference on Data Engineering	1
The VLDB Journal	1
GeoInformatica	1/2
International Conference on Extending DB Technology	1/2
ACM SIGSPATIAL	1/2
Journal of Computer Science and Technology	1/2
Journal of Database Management	1/3
International Symposium on Spatial and Temporal Databases	1/3
International Conference on Mobile Data Management	1/3

When determining the specific model for the corpus training, the LSTM network is chosen based on the following considerations:

(i) In the classification problem, although feedforward networks represented by CNN still have performance advantages, the LSTM can realistically represent or simulate the cognitive process of human behavior, logical development and neural organization, and can handle complex tasks.

(ii) The number of queries in the corpus is not particularly large, and an overly complex deep learning model can not be selected. If a complex model with a large number of parameters is used for training, it is likely to underfit the model. Compared with the Transformer and Bert models proposed recently, the LSTM is simpler and easier to implement.

Table 4 Expert scoring examples

Natural Language Query Examples	Moving Objects Query	Query Type Support	Entity Integrity	Total Score
Where were the trains at a certain time t1?	0.2	0.4	0.4	1
How many trains at Nanjing station and show me the closest way to the station?	0.2	0.15	0	0.35
Is there any one see the red car?	0	0	0	0
What trains pass the station Nantong during [8am, 9qm]?	0.2	0.2	0.4	1
Show me five nearest neighbors to the train 3 right now.	0.2	0.4	0.25	0.85
When did the train pass underground Nanjing Station?	0.2	0.4	0.3	0.9

5.2 Constructing structured languages

After the semantic understanding, we get the query type and map the entities to the corresponding positions according to defined rules. Finally, we combine the suitable operators to form the final structured sentence. The steps are reported in Fig. 5. The template of the query expression is as follows:

query < data relation > *feed filter* [< **special operators** > < values >] *extend* < **special operators** > [< values >] *consume*;

Extracted entities are mapped to the data relation and corresponding values according to the query type. At the same time, operators are selected to constitute the query structure. Some queries with specific objects need to add *filter* [*<Id> = <number>*] or the id of the object to fill in special operators. We support five types of queries: (i) *time interval queries*, (ii) *range queries*, (iii) *nearest neighbor queries*, (iv) *trajectory similarity queries* and (v) *join queries*. To help explain the procedure, we use an example relation storing underground trains as moving objects. The schema is *Trains*(Id: int, Trip: mpoint). Table 5 summarizes kernel operators for processing moving objects.

The system provides structured queries construction and visual query results for different types of moving objects queries. Each query type requires different entities for natural language query, as shown in Fig. 6.

Definition 1 Time interval queries Given an interval $[t_1, t_2]$, the query looks for objects whose time intervals intersect $[t_1, t_2]$.

In order to compare the query time with a trajectory's time, the operator *deftime* is utilized to extract the defined time interval of a trajectory. The operator *filter* evaluates a query

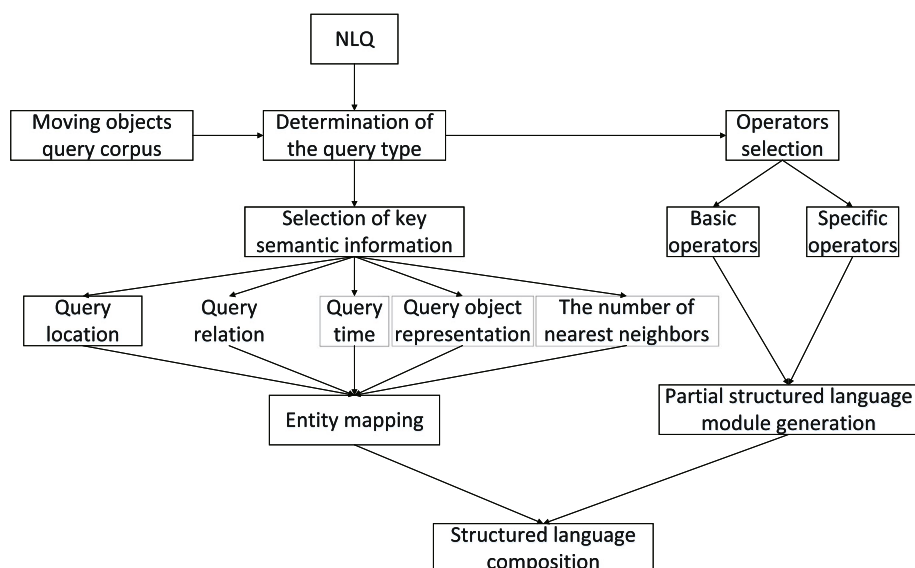


Fig. 5 Building structured language queries

Table 5 Operators for moving objects

Operator	Signature
trajectory	$mpoint \rightarrow line$
atperiods	$mpoint \times periods \rightarrow mpoint$
present	$mpoint \times periods (instant) \rightarrow bool$
deftime	$mpoint \rightarrow periods$
at	$mpoint \times region \rightarrow mpoint$
passes	$mpoint \times region (point) \rightarrow bool$
atinstant	$mpoint \times instant \rightarrow ipoint$
inst	$ipoint \rightarrow instant$
val	$ipoint \rightarrow point$
intersects	$xrectangle \times xrectangle \rightarrow bool$

predicate and returns objects fulfilling the condition. For this query, an intersecting between two time intervals is evaluated. The template of the query expression is as follows in which the content in $\langle \rangle$ will be extracted from natural language expressions.

query $\langle data\ relation \rangle\ feed\ filter[deftime(Trip)intersects\ \langle query\ time \rangle]$
consume;

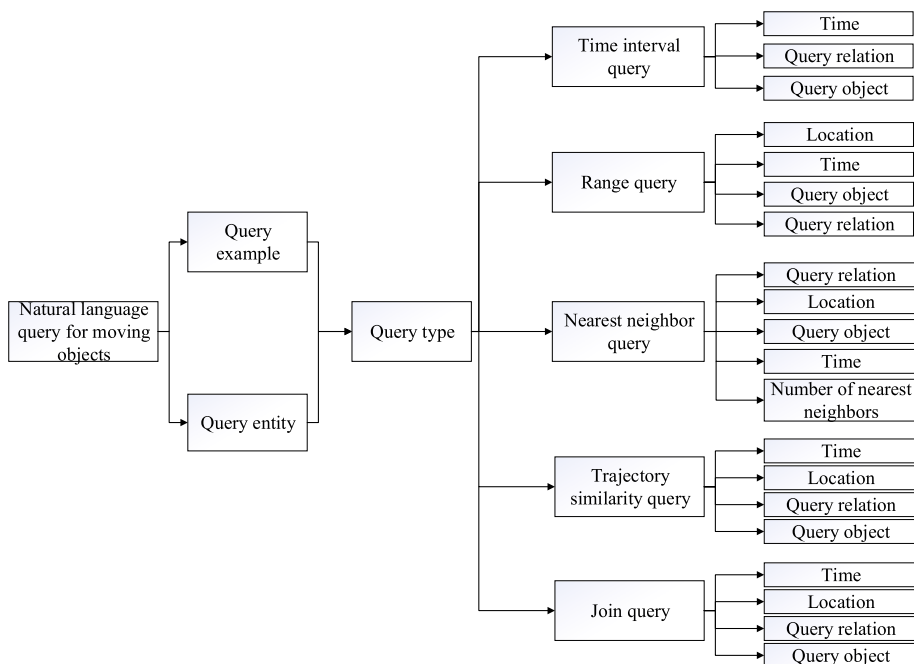


Fig. 6 Query required entities

Assuming the query time is “[7:00, 8:00] on November 20, 2020”, we get the following structured language:

```
query Trains feed filter[deftime(.Trip) intersects[const periods value
("2020 - 11 - 20 - 7 : 00" "2020 - 11 - 20 - 8 : 00" TRUE TRUE)]] consume;
```

There is another approach to accomplish the task by evaluating whether an object is defined at the query time. The operator *atperiod* restricts the movement to a time interval and the operator *present* determines whether an object is valid in a certain time interval. The template is:

```
query < data relation > feed filter[present(.Trip intersects < query
time >)] consume;
```

Definition 2 Spatio-temporal range queries Given a spatio-temporal window Q and a set of trajectories O , the query returns a set of trajectory O' , and for $\forall O' \subset O$, O' intersects $Q(t, r)$.

The special operator *intersect* finds objects that satisfy both time and space conditions. The operator *intersect* can be used to check whether two rectangles cross each other. The operator *deftime* can get the time interval when the moving object is limited in the specific space condition. We use the operation *intersects* to get the intersection between the limited space and the limited time to construct the query. Assuming that the location is “Tiergarten” and the time interval is November 20, 2020 from 8:00 to 10:00, we query trains that meet two conditions above:

```
query Trains feed filter[(deftime(.Trip at Tiergarten)intersects
[const periods value(("2020 - 11 - 20 - 8 : 00" "2020 - 11 - 20 - 10 : 00"
TRUE TRUE))]]]consume;
```

When the query does not specify the specific query object identifier, the structured expression is as follows:

```
query < relation > feed filter [(deftime(. < Trip > at < region >)
intersects < period >)]consume;
```

Definition 3 Nearest Neighbor queries Given a query mq , a distance set D and a relation E with an attribute MO of type *mpoint*, the query which holds all the distances between the queried moving objects returns a subset $E' \subset E$, where each tuple meets the condition that for the tuple $e \in E'$, there exists an instance of time t . The distance $d(e.MO(t), mq(t)) \in D$ is among the k smallest distances.

We use the operator *knearest* for nearest neighbor query. Before querying, we need to create a unit representation of the moving object. The relation *Trains*(*Id*:int, *Trip*:*mpoint*) is used as an example:

```
let UnitsTrains = Trains feed projectextendstream[Id, Line, Up; UTrip :
units(.Trip)]consume;
```

The operator *projectextendstream* creates a unit stream from the *Trip* attribute of each input tuple, and outputs a copy of the original tuple for each unit, which is limited to attributes *Id*, *Line*, *Up* and the new unit attribute *UTrip*. Parameters of the operator *knearest* are ordered

stream of unit tuples, the name of the attribute a , the queried moving object and the number of neighbors k .

The operator *knearest* returns a stream of tuples with the same structure as the input stream. However, the output units in attribute belong to the k nearest moving points represented in the input stream and are limited by the time when objects arrive between the k nearest moving points, and requires the first parameter stream to arrive in the order of units' start time. We create a version of *UnitTrains*, where units are sorted by start time:

```
let UT Ordered = UnitTrains feed extend[Mintime :
minimum(deftime(.UTrip))] sortby[Mintime asc] consume;
```

We can get the final structured language by using *UTOrdered*:

```
query UT Ordered feed filter [(deftime(.UTrip) intersects < period >)]
knearest[UTrip, < object >, < k >]consume;
```

Definition 4 Trajectory similarity queries Given a set of trajectories denoted by O , the query returns the trajectory $o_i \in O$, of which the similarity distance measure m_i is the smallest during the same point t or time interval $[t_1, t_2]$.

We use the operator *atperiod* to limit the movement to a certain period of time, or use the operator *atinstant* to calculate the moving point at a given moment, and return the moving point in a certain period of time or at a certain moment. Then we use the operator *deftime* to project the motion to the time dimension. If we need to limit the number of objects, we can use the operator *head*. Suppose the time interval is from 9:00 to 10:00 on November 20, 2020, we query the subway pairs with cross trajectories, and take the first ten objects for comparison. We use the operator *symmjoin* and the operator *filter* to find out the moving objects that have appeared in a certain period of time and have trajectories crossing, and then project the new line value attributes together with the original partial attributes:

```
query UT Ordered feed filter [(deftime(.UTrip) intersects < period >)]
knearest[UTrip, < object >, 1]consume;
```

Definition 5 Join queries Given a set of trajectories denoted by O , the query returns the trajectory $o_i \in O$, which is shared by the queried moving objects during time interval $[t_1, t_2]$.

If we can find the closest moving object to the query object within a period of time, we can judge that the trajectories of the two moving objects are similar. The operator *knearest* looks for the nearest neighbors in a trajectory segment, but multiple trajectory segments may have appeared in the time interval of a query. We count each nearest neighbor, and the object that has the most frequent occurrence in the time interval is judged to be similar. If the number of occurrences is the same, the longest trajectory has priority. Using *UTOrdered* sorted by start time, we get the final structured language:

```
query Trains feed filter[.Up] extend[Stretch : trajectory(.Trip atperiods
[const periods value ("2003 - 11 - 20 - 9 : 00" "2003 - 11 - 20 - 09 : 10"
TRUE TRUE))]] extend[T : deftime(.Trip)] head[10] t1 Trains feed
filter[.Up]extend[Stretch : trajectory(.Trip atperiods [const periods value
("2020 - 11 - 20 - 9 : 00" "2020 - 11 - 20 - 09 : 10" TRUE TRUE))]]]
```

```

extend[T : deftime(.Trip)] head[10] t2 symmjoin
[.Stretch_t1 intersects1..Stretch_t2] filter[.T_t1 intersects .T_t2] consume;

```

In the expression, the operator relation represents the relation of the query, the operator region represents the limited region, and the operator period represents the limited time period. This type of query does not specify the identity of the specific query object. The expression is as follows:

```

query <relation> feed filter[.<Trip> present <period>] extend[Stretch :
trajectory(. <Trip> atperiods <period>)] head[n] t1 <relation> feed
filter[.<Trip> present <period>] extend[Stretch : trajectory(. <Trip>
atperiods <period>)] head[n] t2 symmjoin [<period> intersects1
<period>] filter[.<period> intersects <period>] consume;

```

6 Experimental evaluation

The evaluation is conducted in a desktop PC (Intel(R) Core(TM) i5-10210U CPU, 1.60 GHz, 8 GB memory, 512 GB disk) running Ubuntu 20.04 (64 bits, GNOME version 3.36.8). We develop the proposal in C/C++ and integrate the implementation into an extensible database system SECONDO. Our system has been developed in a prototype system for translating natural language queries into structured query languages [50].

6.1 Datasets

To the best of our knowledge, there is no benchmark of moving objects queries in natural languages. Therefore, we generate two datasets named Moving Objects Queries (MOQ) and Berlin Trains (BT), respectively.

MOQ contains 280 moving objects queries divided into five categories: (i) time interval queries, (ii) range queries, (iii) nearest neighbor queries, (iv) trajectory similarity queries and (v) join queries. These queries are extracted from more than 60 conference and journal papers in the domain of moving objects and then manually processed by experts. Specifically, we select high-quality queries from the corpus to construct MOQ. MOQ is used to test whether different kinds of moving object queries can get correct structured languages. BT contains moving underground trains and is used to evaluate whether the converted structured sentences can provide expected results for users. Some queries can be seen in the [Appendix](#), and these queries satisfy the diversity of expressions as much as possible.

6.2 Measurement

NALMO is designed for users who are not experts in writing query expressions for moving objects. Three criteria are defined to evaluate the translation quality: (i) *effectiveness*, (ii) *time cost* and (iii) *usability*. Effectiveness mainly contains two measurements: *accuracy* and *precision*.

Effectiveness NALMO is designed for users to conveniently query and obtain query results. Whether executable languages can be generated and whether users can get expected results are important factors. The way to verify the effectiveness is to check whether natural language

expressions are correctly and precisely translated into structured query languages, that is, *accuracy*. The expression of accuracy is shown below.

$$accuracy = \frac{\#executable}{\#overall \ queries} \quad (4)$$

Such a value is defined as the ratio of the number of correctly translated sentences to the overall number of sentences. MOQ is used for test. In the evaluation, domain experts' judgement is considered to determine whether the produced structured query correctly corresponds to the natural language query. However, only using the accuracy is not sufficient to verify the effectiveness of NLIDB systems. This is because users may be not concerned about the generation of structured sentences but pay attention to whether the outputs are the expected results. Motivated by this, we use the *precision* to further evaluate the translation effect. The expression of precision is shown below.

$$precision = \frac{\#expected \ results}{\#overall \ queries} \quad (5)$$

The *precision* is defined as the ratio of the number of queries producing the expected results in the system to the overall number of queries. BT is used for test. In some cases, the natural language expression is unclear or the key information is missing. Therefore, NALMO generates a structured query, but the query fails to output the expected results.

Time cost We do the evaluation by considering (i) *the overall time cost* and (ii) *the time cost in each module*, including parsing, analyzing and constructing the structured languages. We take the average value over three runnings as the final result.

Usability NALMO is designed for ordinary users who do not know structured sentences. User experience is a very important indicator. We recruite four volunteers and conduct three rounds of independent tests to investigate user satisfaction.

Furthermore, when training the model that recognizes the query type, we define a LSTM sequence model. The dimension of the Embedding layer of the model is set to 64, and the number of memory cells of the LSTM layer is set to 128. The output layer of the model is a fully connected layer with five categories.

6.3 Experimental results

6.3.1 Accuracy and precision

We need to conduct a semantic understanding test for each query to determine whether the system can provide the entity information for the query. First, the expert gives a comparison table of the entities contained in each moving objects query, and scores each processed query according to this comparison table. Each query has a full score of 1, and for each missing entity, a certain score will be deducted proportionally. For example, a spatio-temporal range query contains four types of information: time information, location information, query relation and query object. For each missing category of information, a quarter of the point is deducted, which is 0.25 points, and so on. The test is performed according to five query categories, including time period query, range query, nearest neighbor query, trajectory similarity query and join query. We use the average score of each query types as the final results, as shown in Fig. 7.

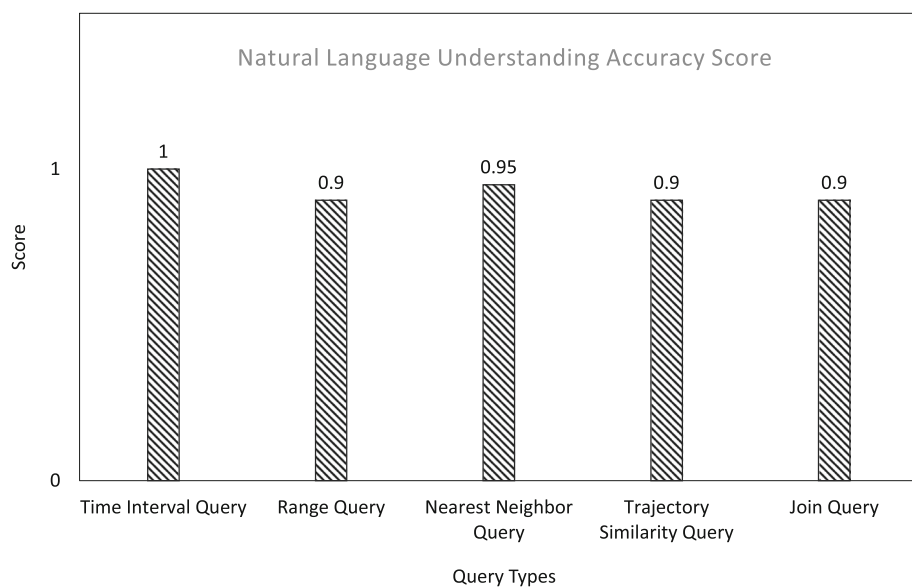


Fig. 7 Natural language understanding accuracy score

In general, different types of scores are basically the same, but in detail, the score of each query category is still different. The time interval query gets the highest score, because the time interval query only needs to concentrate on time information without any other information. We can easily judge and construct time interval query. Other types of queries not only consider time information, but also need to consider other entity information such as location and the number of nearest neighbors.

Figures 8 and 9 show the effectiveness. The overall accuracy of NALMO is 96.8%. Among 9 wrong queries, there are 2 time point queries, 2 time interval queries, 1 spatio-temporal range query, 2 trajectory similarity queries and 2 join queries. The time format of 4 expressions cannot be recognized by spaCy, and NALMO terminates the constructing procedure. There are 3 queries vaguely expressing the time information. Specifically, the word “*right now*”

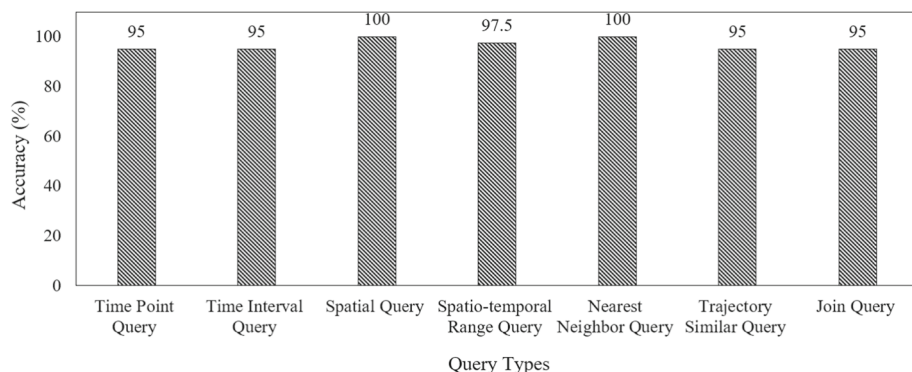


Fig. 8 Accuracy

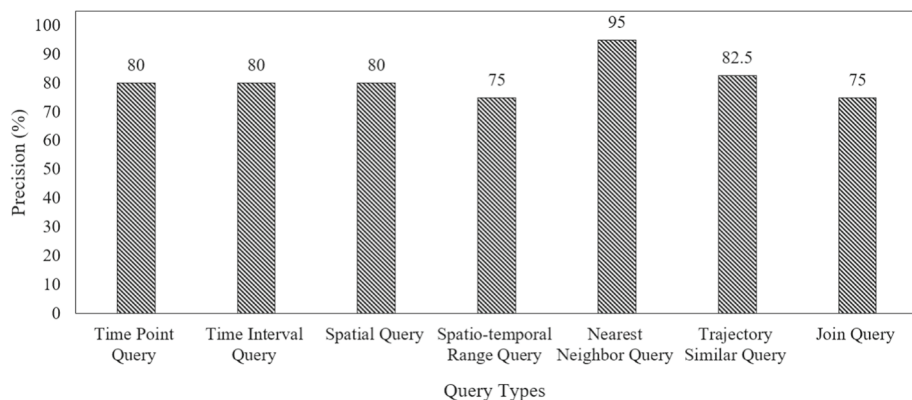


Fig. 9 Precision

in “Where are the trains right now?” cannot be recognized by NALMO as moving objects queries require accurate time information. In trajectory similarity queries, we only support returning the most similar trajectory at the moment but not $k (> 2)$ similar trajectories.

The overall precision is 81.1%. The value is lower than accuracy. This is because some queries are translated into executable structured languages but did not report expected results. Among 53 wrong queries, the time point query contains 8 wrong sentences. The reason is that the relation in the structured sentence is *Planes* which does not exist in the dataset. The time interval query contains 8 wrong sentences among which 4 are spatio-temporal range queries and 2 queries do not find required relations. The spatial query contains 8 wrong sentences. There are 6 queries without locations in the knowledge base such that structured languages are not executable in the database. There are 2 queries looking for non-existent objects. For example, “train G55” in “Where was the train G55 at 12:00 o’clock?” is an object that cannot be queried. The spatio-temporal range query contains 10 wrong sentences among which 6 queries have locations not in the knowledge base and 4 queries do not have required relations. The trajectory similarity query contains 7 wrong sentences. The queries are answered by using algorithms of nearest neighbor queries. The nearest neighbor query contains 2 wrong sentences both of which are identified as similarity queries. The join query contains 10 wrong sentences among which 4 are nearest neighbor queries or trajectory similarity queries, and 4 do not contain suitable locations. The dataset size has a significant impact on the precision of users’ expected query results.

6.3.2 Time cost

In addition, a location-based prefix tree is constructed to improve the extraction efficiency of location information. The location matching of the system is one-to-one matching before. Using the index, the overall efficiency has been greatly improved, especially when the amount of data is relatively large, the efficiency improvement is more obvious. The specific results are shown in Fig. 10.

The time costs are reported in Fig. 11. Nearest neighbor queries, trajectory similarity queries and join queries take more time than other queries, the average time of which are 1.64s, 1.68s and 1.67s, respectively. The three queries consider the time, location, query object identifier, and the number of nearest neighbors, requiring more entities than other queries. The time cost includes two parts: (i) *semantic parsing and analysis*, and (ii) *the*

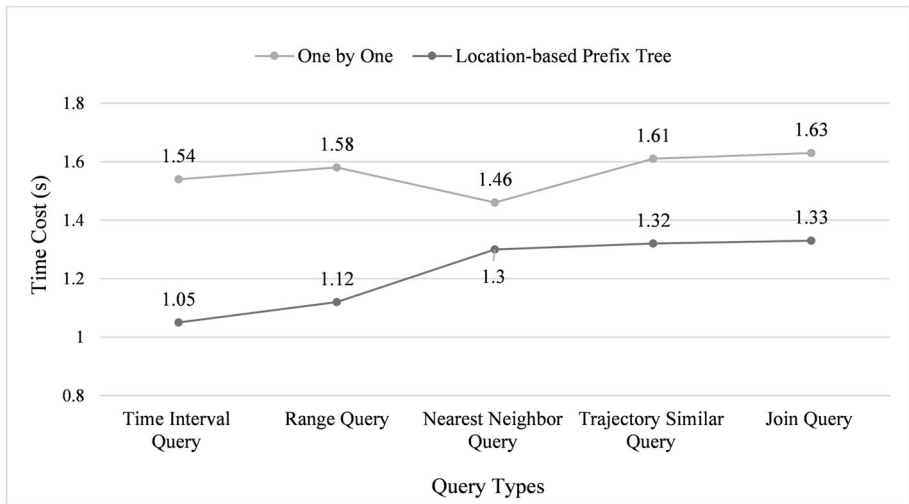


Fig. 10 Natural language understanding time cost

determination of query types and the combination of structured languages. For the seven types of queries in Fig. 11, the average time cost of the entire translation process is about 1.49s, the average time cost of the natural language understanding part is about 1.18s, and the average time cost of the query translation part is about 0.31s. The main part of time is spent on semantic parsing and analysis. The parsing time is affected by the speed of natural language processing tools to a certain extent.

The performance of the overall translation process is evaluated by scaling the size of the dataset. We first extend the *berlintest* database in *SECONDO* by increasing the number of its moving object relations. Then, queries in *MOQ* are divided into five groups: (i) 10 queries

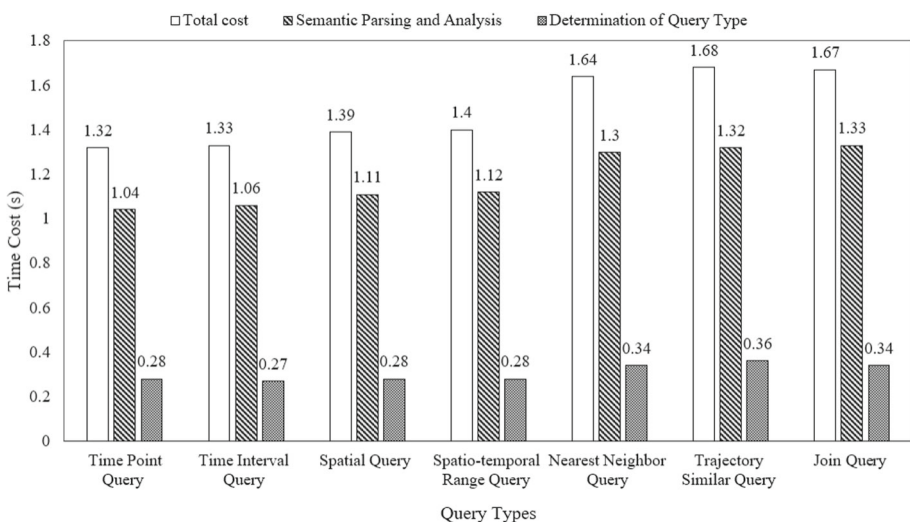


Fig. 11 Time cost

for Group A, (ii) 20 queries for Group B, (iii) 50 queries for Group C, (iv) 100 queries for Group D, (v) 280 queries for Group E. The total time cost of the translation process of each group is measured separately. The experimental results are shown in Fig. 12. The time cost of each group is in the range of 1.49 ± 0.01 s. We conclude that the time cost of the translation process is independent of the size of the test dataset.

6.3.3 Accuracy, precision and time cost between template matching and model

When we use template matching instead of the trained model to determine the query type, accuracy and precision decrease. Time cost is almost the same as before. Table 6 shows that the overall accuracy and precision are 90% and 84.8%, respectively, which are significantly lower than using the trained model. In terms of accuracy, there is no change in time point query and time period query. Spatial query and spatio-temporal range query all add 6 new wrong queries. Although the location is not included in the location knowledge base, the model can still determine the query type and structured queries can be generated. However, template matching lacks location information to directly generate structured queries. The nearest neighbor query, trajectory similarity query and join query add 8, 7 and 4 wrong sentences, respectively. This is because nearest neighbor query, trajectory similarity query and join query are matched by keywords such as “*neighbor*”, “*similar*” and “*same*”. Due to the limitations of template matching, when the keywords do not appear in the sentence, the query cannot be recognized. In terms of precision, there is no change in time point query and time interval query. Spatial query and spatio-temporal range query remain unchanged. This is because the use of model determination does not generate structured sentences, and NALMO will not give user expected results. The wrong queries of nearest neighbor query, trajectory similarity query and join query all do not generate structured sentences. In terms of time cost, there is not much difference compared with using the model which has little effect on user experience.

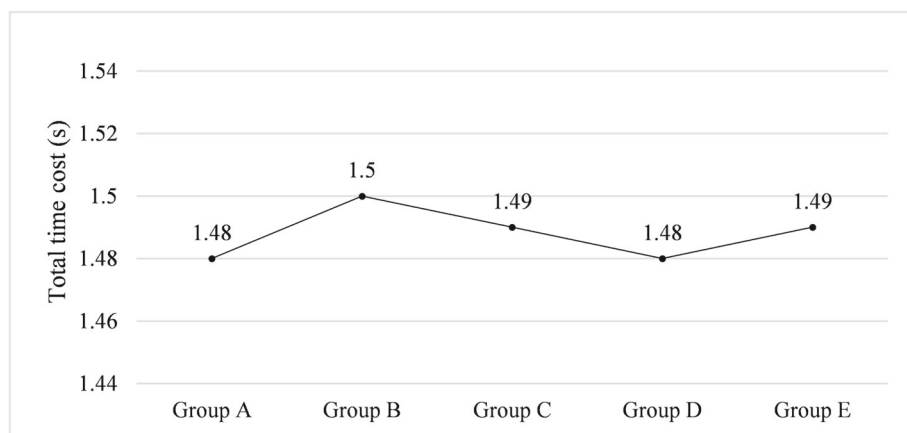


Fig. 12 The total time cost of the entire translation process

Table 6 Accuracy, precision and time cost between template matching and model

Query Types	Accuracy 1 ¹ (%)	Accuracy 2 ² (%)	Precision 1 ¹ (%)	Precision 2 ² (%)	Time Cost 1 ¹ (s)	Time Cost 2 ²
Time Point Query	96.7	96.7	86.7	86.7	1.32	1.26
Time Interval Query	96.7	96.7	86.7	86.7	1.33	1.36
Spatial Query	100	90	86.7	86.7	1.39	1.28
Spatio-temporal Query	98.3	88.3	88.3	88.3	1.4	1.36
Nearest Neighbor Query	100	86.7	96.7	88.3	1.64	1.41
Trajectory Similarity Query	96.7	85	88.3	76.7	1.68	1.49
Join Query	98.3	86.7	86.7	80	1.67	1.47

¹Using the trained model to determine the query type²Using template matching to determine the query type

6.3.4 User interaction

We implement an operator named *nlmo* for the user, and the user can use this operator to translate the corresponding natural language query. This operator takes natural language query as input, and the output of the system has three kinds if the input format is correct. The first is that the structured query statement cannot be constructed and the system directly ends the process; the second is that the system can provide query results for the user but does not meet the user's expectations. The third is the system can accurately give the query results expected by the user. The specific user-system interaction structure is shown in Fig. 13.

In the first case, the system will inform the user of the problem. Generally, there are two types of feedback information: (i) time, location, query object, query relation and other entities which are incorrect or non-existent; (ii) grammatical error and the query type that cannot be determined.

For the first type of feedback information, the entity queried by the user may be not exist, or the entity may be too vague. If the location information is missing, the system allows the user to give feedback to the administrator, and the administrator can add the corresponding location information, such as some personalized POI information that does not exist in the location knowledge base. For the second type of feedback information, the user can only refer to the query example, and then readjust the query to re-query. In the second case, if the user is dissatisfied with the query result, the user can ask the system to regenerate the query result. Three sets of possible query results will be generated inside the system and sorted, and the result with the highest score will be fed back to the user. If the user is not satisfied with the result, the system will give the result with the second score to the user, until all possible results are given. If the user is still dissatisfied with the result, the user has to adjust the query and re-query. If the user selects the result that does not get the highest score, then the result will be re-adjusted as the highest score.

NLIDBs are designed for users who use natural language to query, and we should conduct real user tests. We selected four volunteers for the test, two of them are master students who only understand common natural language queries and do not have any SQL knowledge. The other two are master students studying moving objects databases. They are not only familiar with the way natural language queries are proposed, but also the structured queries of moving objects, and can independently construct structured query sentences for database. Four volunteers need to participate in three tests separately: (i) The user uses the provided query to test and give feedback on the query; (ii) Based on the query used in the previous test, the user independently proposes a new query and gives feedback; (iii) The user interactively revises the query made by himself and gives feedback again. Specifically, in the first test, a total of 120 queries based on different types of queries will be given, that is, 30 queries of

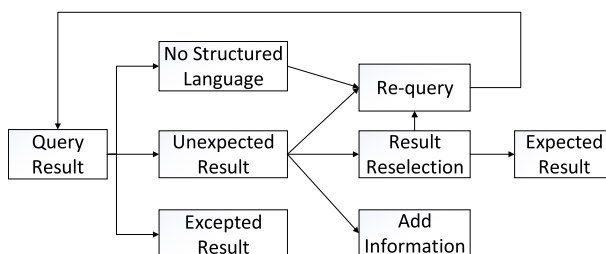


Fig. 13 User-system interaction structure

each type for users to test. These queries can get structured sentences. Ordinary volunteers only need to give a judgment whether each query meets the expectations, and the expert volunteers not only need to give the judgment of the query but also give the reason for the error. In the second test, four volunteers are required to give queries that are different from the test examples. About 20 queries of each type are given, for a total of 80 queries, and we perform corresponding query tests. Similarly, ordinary volunteers need to give a judgment whether each query meets the expectations, and the expert volunteers not only need to give the judgment of the query but also give the reason for the error. In the last test, according to the query that users cannot get the expected result in the first and second tests, users need to interact with the system. Users can add certain additional entities to the database or let the system regenerate a new query. In the end, the volunteers give feedback on satisfaction of using the system. In addition, each volunteer is tested individually in an independent space, and each volunteer cannot communicate before completing all the tests, ensuring that the volunteers will not affect each other. Volunteers researching moving objects databases may construct high-quality and robust moving objects queries due to their inherent knowledge, and the queries may be quite different from those constructed by ordinary volunteers. Therefore, ordinary volunteers and expert volunteers are selected for comparison, to avoid subjective differences due to inherent knowledge.

From the previously constructed moving objects corpus, we get 120 robust natural language queries for volunteers to test through the selection of experts. At the same time, the moving objects queries proposed by users in the second round of testing are also used as the test dataset. The two test datasets are slightly different. We specifically screen out the queries proposed by ordinary volunteers and the queries extracted from the papers and journals as examples for comparison. The queries proposed by users and the given queries are basically the same in the natural language query construction of different moving objects. However, the queries proposed by volunteers, especially ordinary volunteers, are slightly different from the expected queries supported by the system. For example, the query relation and query object in the sample query are very clear and exist in the dataset, but sometimes the query object and query relation given by ordinary volunteers are rather ambiguous, as shown in Table 7.

The experiment conduct a separate analysis for each round of testing. We show the results of each round of testing. In the first round of testing, both ordinary volunteers and expert volunteers give great satisfaction, and each category has great satisfaction. The satisfaction of ordinary volunteers and expert volunteers are 92.5% and 98.6% respectively, which are almost

Table 7 Query comparison

Sample Query	Volunteer Query
Where did the train 123 go at 8am?	Where did the train 24 go from 8am to 9am?
Find the trains that pass the place Tiergarten from 8:50 to 11:30.	Which buses went to the Suzhou station from 8:05 to 10:00 o'clock?
Tell me the nearest neighbor to the train 1 between 9:00 and 9:01 o'clock.	Could you find the 12 nearest neighbors of the green train during [2:45, 3:56]?
Would you mind telling me the trajectory that is similar to the fastest train 134 this afternoon from 7pm to 9pm?	Can you show me two trains that are similar to the train 3 that passes People's Square?
Please list the common stations that train 12 and Train 67 pass through from 8:00am to 10:05am.	Could you help me list all the same kindergartens that bus 23 and bus 22 passed between 9 am and 11 am on March 10th?

the same, because these test cases are extracted from high-quality journals and conference papers. Besides, the test cases are constructed through professional modification and the results are within expectations. Among the queries that do not meet users' expectations, expert volunteers give corresponding reasons. For example, the query object does not exist or the database does not support a certain type of query object. The reasons of the errors given by the expert volunteers are reasonable. The second round of testing is more challenging. We need to pay attention to whether the inherent knowledge of the expert will affect the construction of the query, but the results show that there is no big gap between ordinary volunteers and expert volunteers. The main reason is that the first round of testing gives ordinary volunteers a certain understanding of the query, and they will not propose vague queries. However, ordinary volunteers do not know the query types of moving objects, and the query types are not evenly distributed. The distribution of the queries proposed by ordinary volunteers and expert volunteers is shown in the following table. Similarly, several volunteers test their queries and give corresponding feedback. The queries proposed by the expert volunteers are relatively standardized and can be well translated into structured languages. Two of the queries do not get the expected results because the query relation does not exist. However, three queries proposed by ordinary volunteers show vague semantics, the system can not construct structured sentences, and the three queries do not meet the expectations of volunteers. In the third round of testing, volunteers can improve the unsatisfactory queries in the first and second rounds. If the reason is POI and other locations do not exist, every volunteer can make suggestions to expand the location knowledge base. Volunteers who do not get expected results can ask the system to regenerate the results. In the end, all volunteers give reasons for unsatisfactory results so that we can optimize the system. The specific results are shown in Table 8. One query proposed by ordinary volunteers does not meet expectations, because the system judges the nearest neighbor query as a trajectory similarity query, and the volunteers can not get the expected results even if they ask the system to regenerate the results. For queries with missing locations, users can get the expected results after adding locations.

7 System development

We design and develop NALMO, a natural language query translation system for moving objects. The system is integrated in the scalable database system SECONDO. The final form is an algebra module in SECONDO, and the algebra implements an operator named *nlmo*. Users can use this operator to translate the corresponding natural language query on the database in the SECONDO system, and then execute the translated executable query to obtain the query result.

Table 8 Volunteer Satisfaction

Volunteer	First Round Satisfaction	Second Round Satisfaction	Third Round Satisfaction
Ordinary Volunteer1	115/120 (95.8%)	72/80 (90%)	12/13 (92.3%)
Ordinary Volunteer2	116/120 (96.7%)	70/80 (87.5%)	13/14 (92.9%)
Expert Volunteer1	118/120 (98.3%)	78/80 (97.5%)	4/4 (100%)
Expert Volunteer2	119/120 (99.2%)	77/80 (96.3%)	4/4 (100%)

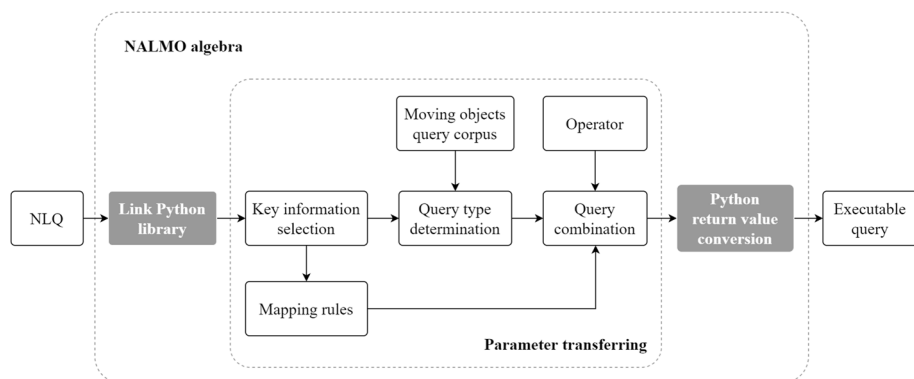


Fig. 14 The system development architecture

There are three main steps of system development: (i) use Python to implement the functions of the semantic understanding part and the query translation part; (ii) create an algebra that implements the functions of NALMO system; (iii) activate and compile the algebra for normal use of the operator *nlmo*. The architecture of the system development is shown in Fig. 14. In this process, query processing in natural language understanding is programmed in Python, while structured statement generation is implemented based on SECONDO and programmed in C++. Therefore, the main challenge in system development is the processing from Python modules to C++ modules. The final solution is to use the C++ library provided by Python to call the semantic understanding and query translation modules in the C++ program, mainly including (i) *the linking of the Python library*, (ii) *the transfer of parameters* and (iii) *the conversion of Python return values*.

The linking of the Python library Header files and library files are already included in the Python installation directory. We need to link to this library before using the Python module. We can call the Python statement directly, and then we can use the module functions in the C++ library provided by Python for initialization. Finally, we can use the corresponding functions to directly run Python statements or call the required functions. After encapsulating the natural language understanding module for moving objects query into a C++ module, we can use C++ to run the natural language understanding script file and call the functions.

The transfer of parameters We need to use C++ to pass parameters to Python modules, that is, pass parameters when calling functions in Python modules. The arguments of Python are actually tuples, and passing an argument is actually constructing a suitable tuple.

The conversion of Python return values We need to switch the working directory of Python to the path where the module is located, and load the module by the module name instead of the file name. Module loading or function loading needs to be verified successfully, otherwise the process may cause a stack error and program crashing. We should use suitable functions to dereference the object for Python garbage collection.

8 Conclusion

We design and develop a natural language interface for moving objects databases. The proposal supports translating five types of queries. Experiment results demonstrate the advantage

of our method in terms of accuracy, precision, efficiency and usability. An important task in the future is to further improve the fault tolerance and the association prompts of input errors. When the input is not successfully sent into the system, NALMO is able to determine whether the user has entered similar information before. Considering that we can use the index to improve moving objects query efficiency, the corresponding index structure needs to be introduced in the future. The moving objects database has many index structures, such as R-Tree, TPR-Tree, SETI and other indexes. Supporting these indexes during the construction of structured language is a challenge task.

Acknowledgements This work is supported by National Natural Science Foundation of China (61972198), Natural Science Foundation of Jiangsu Province of China (BK20191273).

Author Contributions Conflicts of reviewers: Ralf Hartmut Güting, rhg@fernuni-hagen.de is the Ph.D advisor of Author Jianqiu Xu. Hua Lu, luhua@ruc.dk, Roskilde University is the co-author of the paper. Zhifeng Bao, zhifeng.bao@rmit.edu.au, RMIT University has the co-authorship with Jianqiu Xu. Ouri E. Wolfson, wolfson@uic.edu, University of Illinois at Chicago has the co-authorship with Jianqiu Xu. Yu Zheng, msyuzheng@outlook.com, JD Finance has the co-authorship with Jianqiu Xu. Bin Yao yaobin@cs.sjtu.edu.cn, Shanghai Jiao Tong University has the co-authorship with Jianqiu Xu.

Data Availability The datasets generated and analysed during this study are available at: <https://pan.nuaa.edu.cn/share/3208e9a270de8389f9477d1cda> or <https://pan.baidu.com/s/1TVTPPZmDQ4nT8E7wXOXaAA?pwd=3qjc>.

Appendix

Table 9 Moving objects queries in natural language

Id	Query Expression
Q1	Which places did the trains go between 9:00 and 10:00 o'clock?
Q2	Which trains pass the station Mehringdamm from 8:00 to 10:00?
Q3	Please tell me the position of train 3 between 11am and 1pm?
Q4	Who can tell me which train has the similar trajectory of the train 134 at 7am?
Q5	Which trains are the 5 continuous nearest neighbors of the train 345 from 8:00 to 9:00 o'clock?
Q6	Did any trains pass Westhafen between 1pm and 1:30pm?
Q7	Find the trains that pass Mehringdamm from 8:20 to 10:59.
Q8	Which train is the nearest neighbor of the train 123 from 9am to 10am?
Q9	Which train is the nearest neighbor to the train 345 between [9am, 10am]?
Q10	Could you tell me which train is similar to the train 77 during [5pm, 9pm]?
Q11	What is the location of the train 1 from 8:00 to 9:10?
Q12	Please find almost 20 continuous nearest neighbors of the train 28 which is in Nanjing between 2am and 4am.
Q13	Which trains arrive at the Zoogarten between 8:18 and 10:27?
Q14	May I know the similar trajectory of the taxi 2 from 1:13 to 2:24?
Q15	What is the location of train 44 at 23:01?
Q16	Do you know the route of the train 56 between 7:00am and 16:00pm?
Q17	Query the trains that have left from 11am to 2pm.
Q18	Find the periods that the train 11 went to Alexanderplatz.

Table 9 continued

Id	Query Expression
Q19	Did any trains pass the station Alexanderplatz during [11am, 12am] today?
Q20	Please tell me which trains have been to the Mehringdamm station from 8:20 to 10:00?
Q21	Find precisely seven closest neighbors of the train 8 during [10am, 6pm].
Q22	Which train has the similar trajectory of the train 17 between 17:00 and 19:00 o'clock?
Q23	Would you mind telling me the trajectory that is close to the fastest train 29 at 18:19 in Beijing?
Q24	May I know which train is similar to the train 1 that passes Sydney Opera House near me?
Q25	Tell me which train has the similar trajectory of the train 37 at Shanghai station during [7:19, 11:19].
Q26	Please tell me which trains have been to the Koepenick from 8:20 to 10:00?
Q27	Which train is the nearest neighbor to the train 345 between [9am, 10am]?
Q28	Where was the train 5 at 17:00 o'clock on Saturday?
Q29	Find the periods that train 17 passed by the Arch of Triumph?
Q30	From 8am to 9am, which trains are five nearest neighbors to the train 47?
Q31	Among these trains, please list at least three trains that can reach Beijing from 9:00 to 9:20.
Q32	Find the same driving path for train1 and train23 from 8:00 to 14:00.
Q33	Show the same path of train2 and train4 between 6:40am and 10:15am.
Q34	Please help me determine the common trajectory of train 13 and train 2 between 5pm and 7pm that pass Beijing Station.
Q35	Can you help me find out exactly what the same trajectory of the three trains from 10:00am to 14:00pm?

References

- Affolter K, Stockinger K, Bernstein A (2019) A comparative survey of recent natural language interfaces for databases. *VLDB J* 28(5):793–819
- Amsterdamer Y, Kukliansky A, Milo T (2015) Nl2cm: A natural language interface to crowd mining. In: *ACM SIGMOD*, pp 1433–1438
- Artzi Y, Zettlemoyer LS (2011) Bootstrapping semantic parsers from conversations. *EMNLP* 2011:421–432
- Baik C, Jagadish HV, Li Y (2019) Bridging the semantic gap with SQL query logs in natural language interfaces to databases. *ICDE* 2019:374–385
- Booth J, Eugenio BD, Cruz IF, Wolfson O (2015) Robust natural language processing for urban trip planning. *Appl Artif Intell* 29(9):859–903
- Chakka VP, Everspauth A, Patel JM (2003) Indexing large trajectory data sets with SETI. In: *CIDR*
- Cheng J, Reddy S, Saraswat VA, Lapata M (2017) Learning structured natural language representations for semantic parsing. *ACL* 2017:44–55
- Chon HD, Agrawal D, Abadi AE (2003) Range and knn query processing for moving objects in grid model. *Mob Networks Appl* 8(4):401–412
- Damljanovic D, Agatonovic M, Cunningham H (2011) Freya: An interactive way of querying linked data using natural language. *ESWC, Lecture Notes in Computer Science* 7117:125–138
- Deutch D, Frost N, Gilad A, Haimovich T (2018) Nlprovenans: Natural language provenance for non-answers. *Proc VLDB Endow* 11(12):1986–1989
- Deutch D, Frost N, Gilad A (2020) Explaining natural language query results. *VLDB J* 29(1):485–508
- Dong L, Lapata M (2018) Coarse-to-fine decoding for neural semantic parsing. *ACL* 2018:731–742
- Ferré S (2017) Sparklis: An expressive query builder for SPARQL endpoints with guidance in natural language. *Semantic Web* 8(3):405–418
- Giordani A, Moschitti A (2012) Translating questions to SQL queries with generative parsers discriminatively reranked. In: *COLING*, pp 401–410

15. Gur I, Yavuz S, Su Y, Yan X (2018) Dialsq: Dialogue based structured query generation. In: ACL, pp 1339–1349
16. Gü R, Almeida Vd, Ding Z (2006) Modeling and querying moving objects in networks. *Vldb Journal* 15(2):165–190
17. Güting RH, Schneider M (2005) *Moving Objects Databases*. Elsevier
18. Güting RH, Behr T, Düntgen C (2010) SECONDO: A platform for moving objects database research and for publishing and integrating research implementations. *IEEE Data Eng Bull* 33(2):56–63
19. Güting RH, Behr T, Xu J (2010) Efficient k-nearest neighbor search on moving object trajectories. *Vldb J* 19(5):687–714
20. Iyer S, Konstas I, Cheung A, Krishnamurthy J, Zettlemoyer L (2017) Learning a neural semantic parser from user feedback. In: ACL, pp 963–973
21. Jensen C, Lu H, Yang B (2009) Graph model based indoor tracking. In: MDM, pp 122–131
22. Jia Z, Abujabal A, Roy RS, Strötgen J, Weikum G (2018) TEQUILA: temporal question answering over knowledge bases. In: ACM CIKM, pp 1807–1810
23. Jia Z, Pramanik S, Roy RS, Weikum G (2021) Complex temporal question answering on knowledge graphs. In: CIKM '21, pp 792–802
24. Kamath A, Das R (2019) A survey on semantic parsing. In: AKBC
25. Kim J, Hong D, Kang H, Han K (2006) TMOM: A moving object main memory-based DBMS for telematics services. In: W2GIS 2006, Lecture Notes in Computer Science, vol 4295, pp 259–268
26. Klein D, Manning CD (2003) Accurate unlexicalized parsing. In: ACL, pp 423–430
27. Li F, Jagadish HV (2014) Constructing an interactive natural language interface for relational databases. *Proc VLDB Endow* 8(1):73–84
28. Li J, Wang W, Ku W, Tian Y, Wang H (2019) Spatialnli: A spatial domain natural language interface to databases using spatial comprehension. In: ACM SIGSPATIAL, pp 339–348
29. Li Y, Rafiei D (2017) Natural language data management and interfaces: Recent development and open challenges. *SIGMOD 2017*:1765–1770
30. Li Y, Rafiei D (2018) *Natural Language Data Management and Interfaces*. Synthesis Lectures on Data Management 10(2):1–156
31. Li Y, Yang H, Jagadish HV (2007) Nalix: A generic natural language search environment for XML data. *ACM Trans Database Syst* 32(4):30
32. Liu P, Wang M, Cui J, Li H (2021) Top-k competitive location selection over moving objects. *Data Sci Eng* 6(4):392–401
33. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: NIPS, pp 3111–3119
34. Miller GA (1995) Wordnet: A lexical database for english. *Commun ACM* 38(11):39–41
35. Mokbel MF, Sakr MA, Xiong L, Züfle A et al (2022) Mobility data science (dagstuhl seminar 22021). *Dagstuhl Reports* 12(1):1–34
36. Pasupat P, Liang P (2015) Compositional semantic parsing on semi-structured tables. *ACL 2015*:1470–1480
37. Pelekis N, Frentzos E, Giatrakos N, Theodoridis Y (2008) HERMES: aggregative LBS via a trajectory DB engine. pp 1255–1258
38. Pfoer D, Jensen CS, Theodoridis Y (2000) Novel approaches in query processing for moving object trajectories. In: VLDB, pp 395–406
39. Poon H, Domingos PM (2009) Unsupervised semantic parsing. In: EMNLP 2009:1–10
40. Popescu A, Etzioni O, Kautz HA (2003) Towards a theory of natural language interfaces to databases. In: IUI, pp 149–157
41. Popescu A, Armanasu A, Etzioni O, Ko D, Yates A (2004) Modern natural language interfaces to databases: Composing statistical parsing with semantic tractability. In: COLING, pp 141–147
42. Price PJ (1990) Evaluation of spoken language systems: the ATIS domain. In: *Speech and natural language*, pp 91–95
43. Saha D, Floratou A, Sankaranarayanan K, Minhas UF, Mittal AR, Özcan F (2016) ATHENA: an ontology-driven system for natural language querying over relational data stores. *Proc VLDB Endow* 9(12):1209–1220
44. Stratica N, Kosseim L, Desai BC (2005) Using semantic templates for a natural language interface to the CINDI virtual library. *Data Knowl Eng* 55(1):4–19
45. Suhr A, Iyer S, Artzi Y (2018) Learning to map context-dependent sentences to executable formal queries. In: NAACL-HLT, pp 2238–2249
46. Tang LR, Mooney RJ (2001) Using multiple clause constructors in inductive logic programming for semantic parsing. *EMCL, Lecture Notes in Computer Science* 2167:466–477

47. Tao Y, Papadias D (2001) Mv3r-tree: A spatio-temporal access method for timestamp and interval queries. In: VLDB, pp 431–440
48. Ulm G, Smith S, Nilsson A, Gustavsson E, Jirstrand M (2021) OODIDA: on-board/off-board distributed real-time data analytics for connected vehicles. *Data Sci Eng* 6(1):102–117
49. Vazirgiannis M, Theodoridis Y, Sellis TK (1998) Spatio-temporal composition and indexing for large multimedia applications. *Multim Syst* 6(4):284–298
50. Wang X, Xu J, Wang Y (2020) NLMO: towards a natural language tool for querying moving objects. *MDM* 2020:228–229
51. Wang X, Xu J, Lu H (2021) NALMO: A natural language interface for moving objects databases. *SSTD* 2021:1–11
52. Wang Y, Berant J, Liang P (2015) Building a semantic parser overnight. *ACL* 2015:1332–1342
53. Wong YW, Mooney RJ (2006) Learning for semantic parsing with statistical machine translation. In: HLT-NAACL, pp 439–446
54. Xiong X, Elmongui HG, Chai X, Aref WG (2007) Place: A distributed spatio-temporal data stream management system for moving objects. *MDM* 2007:44–51
55. Xu J, Güting RH (2013) A generic data model for moving objects. *GeoInformatica* 17(1):125–172
56. Yaghmazadeh N, Wang Y, Dillig I, Dillig T (2017) Sqlizer: query synthesis from natural language. *Proc ACM Program Lang* 1(OOPSLA):63:1–63:26
57. Yin P, Neubig G (2017) A syntactic neural model for general-purpose code generation. *ACL* 2017:440–450
58. Yuan H, Li G (2021) A survey of traffic prediction: from spatio-temporal data to intelligent transportation. *Data Sci Eng* 6(1):63–85
59. Zenz G, Zhou X, Minack E, Siberski W, Nejd W (2009) From keywords to semantic queries - incremental query construction on the semantic web. *J Web Semant* 7(3):166–176
60. Zheng W, Cheng H, Zou L, Yu JX, Zhao K (2017) Natural language question/answering: Let users talk with the knowledge graph. *CIKM* 2017:217–226
61. Zheng Z, Li F, Huang M, Zhu X (2010) Learning to link entities with knowledge base. In: HLT-NAACL, pp 483–491
62. Sen J, Lei C, Quamar A, Ozcan F, Efthymiou V, Dalmia A, Stager G, Mittal AR, Saha D, Sankaranarayanan K (2020) ATHENA++: natural language querying for complex nested SQL queries. *Proc VLDB Endow* 13(11):2747–2759

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.